Room Header Buttons Ordering

Google Summer Of Code 2026 - Proposal

Field	Details
Name	Rahian da Silva Santos
Email	rahiansilvasantos@gmail.com
GitHub	github.com/TheRazorbill
LinkedIn	linkedin.com/in/rahian/
Rocket.Chat Username	@rahian.santos
Timezone	UTC-3
University	Cruzeiro do Sul
Degree Program	Computer Science
Organization	Rocket.Chat
Project Size	90 hours

Rocket.Chat Mentors:

Martin Schoeler | Milton Rucks

23th March, 2026

Summary

Today, Room Header actions (Threads, Search, Call, Pins, etc.) are rendered through Room Toolbox with fixed layout rules (featured + up to 6 visible buttons + overflow). This makes it hard for workspaces to prioritize critical actions and reduces header flexibility according to organizational needs.

I propose implementing a configurable layout engine for Room Toolbox actions, allowing administrators to define ordering and pinning (featured) at workspace level, with safe rollout through Feature Preview (per user).

The solution includes:

- versioned JSON setting configuration
- frontend setting consumption with useSetting
- dynamic administration UI updates with useSettingStructure
- unit and E2E tests with Playwright/axe
- Storybook stories for impacted components

Problem Description

In Rocket.Chat, the Room Header combines inline elements (ex.: `Favorite`, `Encrypted`, `Translate`, `Topic`) with header action buttons, which are managed by **Room Toolbox**.

Room Toolbox receives actions through hooks (`roomActionHooks`) and also supports actions from apps/plugins. Each action has a default `order`, but the current layout applies rigid rules:

- featured actions are displayed first
- when expanded, at most 6 normal actions are shown in the main bar
- the remaining actions go to the overflow menu ("..."), including app actions

Limitations:

- **Low workspace-level configurability:** administrators cannot explicitly define what stays pinned in the bar or reorder buttons according to workspace priorities
- **Clutter and discoverability issues:** important actions may be pushed into overflow by fixed rules, reducing user efficiency

- **Rollout and validation challenges:** header behavior changes require deployment and can cause regressions; a preview/rollout flow aligned with the existing Feature Preview system is preferred

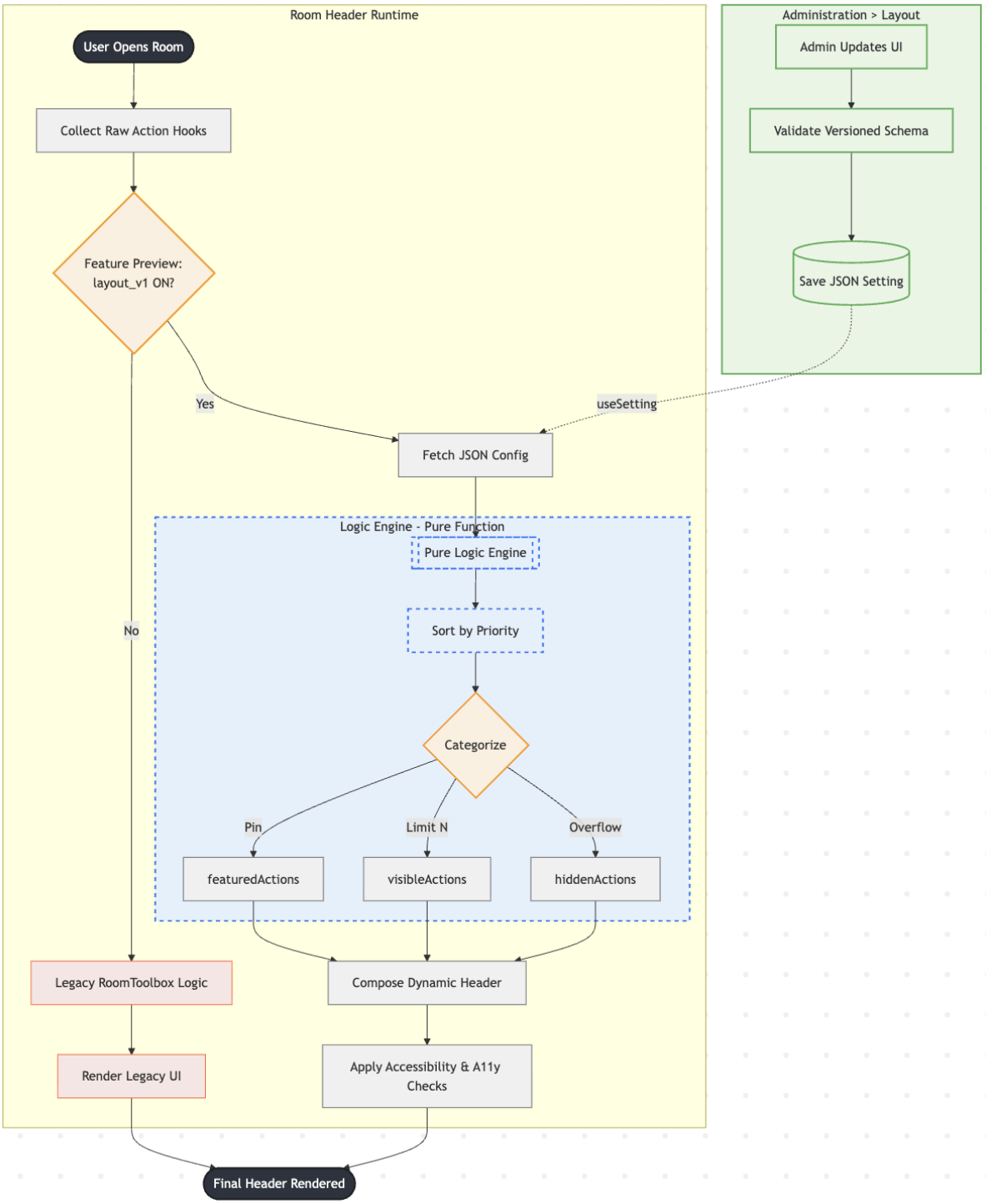
Proposed Solution: Hybrid Dynamic Layout Engine

I propose the implementation of a deterministic, schema-driven layout engine to orchestrate Room Header actions. The solution is built upon four strategic pillars:

- **Versioned Configuration Schema:** A workspace-level JSON setting that governs action priority and "Pinned" (featured) status. By using a versioned schema, we ensure future-proofing and easy migrations.
- **Pure Logic Engine:** A standalone, typed layout processor that transforms raw action hooks into three distinct buckets (*Featured*, *Visible*, and *Hidden*) based on the admin's configuration or a legacy fallback.
- **Dynamic Administration UI:** A dedicated management interface under `Administration > Settings > Layout`, utilizing `useSettingStructure` to ensure the UI remains reactive and synchronized with the underlying JSON schema.
- **Progressive Rollout via Feature Previews:** Integration with the `/account/feature-preview` system, allowing a risk-free, per-user opt-in period to gather feedback before global activation.

Flowchart in Mermaid

High-level architecture showing the orchestration between admin-defined configurations, the deterministic layout engine, and the per-user rollout strategy.



Component 1: Typed Model + Layout Engine (Pure Functions)

Goal: transform `actions[] + config` into three final lists:

- `featuredActions[]` (*pinned*)
- `visibleActions[]` (up to N, with N=6 as *baseline*)
- `hiddenActions[]` (*overflow*)

Conceptual Model:

- `RoomToolboxLayoutConfig` with `version` and `items[]` (id + featured + optional visible).
- Deterministic rules:
  - featured always first
  - then sort by config (or current order as default)
  - keep overflow behavior explicit (remaining actions in the "..." menu)
  - safely handle unknown actions (new actions)

Technology:

- TypeScript + pure functions to maximize Jest testability

Component 2: Admin Configuration (workspace-wide)

Goal: allow administrators to define Room Toolbox ordering/pinning per workspace.

Approach:

- store configuration in a versioned JSON setting
- expose configuration under Administration > Settings > Layout > Room Header to keep consistency with the existing panel
- build the UI with Fuselage components, including:
  - Table for action listing
  - Button for move up/down controls (v1: accessible and simple)
  - Icon for visual affordances
  - SortableList for future evolution (optional drag-and-drop)
- allow marking actions as pinned/featured
- save and validate config

Settings hooks:

- useSetting to consume JSON configuration in header rendering
- useSettingStructure to ensure admin UI dynamically reflects setting structure changes without hardcoded assumptions

Compatibility:

- app/plugin actions remain functional
- v1 may keep app actions in overflow by default, with documented behavior

Component 3: Feature Preview integration (per-user)

Goal: controlled rollout of the new behavior without affecting all users at once.

Approach:

- Add a Feature Preview flag (ex.: `room_toolbox_layout_v1`) in `/account/feature-preview` (user preference).
- When the flag is ON for a user:
  - use *layout engine + workspace configuration*
- When OFF:
  - keep legacy behavior

Component 4: Code Touchpoints and Technical Integration

Main files/areas to update:

- app/ui-utils/client/lib/RoomToolbox.ts - action registration/composition logic - layout/ordering integration point
- client/views/room/Header/Toolbox/ - final action rendering (featured, visible, overflow) - connection with Feature Preview state and layout setting

These touchpoints separate responsibilities between:

- action collection/registration
- pure layout rule application
- header rendering

Component 5: Testing + Storybook (Quality and maintainability)

Goal: ensure stability, minimize regressions, and simplify review.

- Unit tests (Jest/RTL):
  - *layout engine* (*ordering/pinning/overflow* rules)
  - UI *admin* (reordering/pinning + persistence)
- E2E (Playwright):
  - *toggle preview* in `/account/feature-preview`
  - validate that header reflects ordering/pinning for users with preview enabled
- Accessibility (axe):
  - `/account/feature-preview` and UI with no critical violations
- Storybook:
  - stories for impacted components (toolbar/overflow and configuration UI)

Functional Requirements (MVP)

As a workspace administrator:

- I want to reorder Room Header actions to reflect my team operational priorities
- I want to mark actions as pinned/featured so they remain visible in the main bar
- I want configuration to be persisted in a versioned and safe format
- I want setting structure changes to be reflected dynamically in the admin UI

As an end user:

- I want to keep the current experience when preview is disabled
- I want to test the new layout through Feature Preview without affecting other users

As the system:

- it must keep deterministic layout behavior (featured, visible, overflow)
- it must preserve compatibility with unknown actions and app/plugin actions
- it must provide a safe fallback to legacy behavior when preview is disabled

Scope and Boundaries

In Scope (MVP)

- typed layout engine with pure, predictable rules
- versioned JSON setting for workspace-wide configuration
- administration UI under Administration > Settings > Layout > Room Header
- setting consumption with useSetting and structural updates with useSettingStructure
- per-user Feature Preview integration for progressive rollout
- unit, E2E, and accessibility test coverage

Out of Scope (MVP)

- complete Room Header redesign beyond action ordering/pinning
- full replacement of current overflow mechanics
- mandatory advanced drag-and-drop in v1 (can be a future enhancement)
- complete custom ordering policy for all app actions in first delivery

Risks and Mitigation

- **Risk**: Visual/behavior regressions in header. **Mitigation Strategy**: Pure-function layout engine + deterministic ordering/pinning/overflow tests.
- **Risk**: UX breakage for existing users. **Mitigation Strategy**: Per-user Feature Preview + explicit fallback to legacy behavior.
- **Risk**: Inconsistency between settings and admin UI. **Mitigation Strategy**: useSettingStructure to reflect changes dynamically.
- **Risk**: Compatibility with unknown/app actions. **Mitigation Strategy**: Documented fallback rules and edge-case coverage.
- **Risk**: E2E flakiness. **Mitigation Strategy**: Stable Playwright scenarios and objective acceptance criteria.

Rollout Strategy

- Phase 1 (Technical Shadow/Preview): enable engine for users with Feature Preview ON, keep legacy behavior for others
- Phase 2 (Guided Validation): collect UX feedback, adjust compatibility rules, and close testing gaps
- Phase 3 (Expansion): broaden adoption after regression and accessibility indicators stabilize

Operational fallback:

- if regression occurs, keep flag OFF for affected users and preserve legacy flow

Success Criteria and Measurement Method

- Functional quality: 100% of critical layout scenarios (featured/visible/hidden) validated by unit tests
- Integration quality: ON/OFF Feature Preview flow validated by E2E with no critical regressions
- Accessibility: zero critical violations on impacted screens (admin + feature preview + header) in axe checks
- Stability: reduction in reported ordering/pinning bugs during validation phase

Method:

- compare legacy behavior vs new engine in equivalent scenarios
- validate scenarios with core, unknown, and app/plugin actions
- use automated test evidence and mentor reviews for incremental acceptance

Community Benefits

- Rocket.Chat: reduces header clutter, improves efficiency, and allows workspaces to prioritize key actions
- Maintainability: centralized pure and typed layout rules simplify review and evolution
- Quality: strong test coverage + Storybook stories accelerate feedback and reduce regressions
- Accessibility: axe/Playwright-driven workflow aligned with current project practices

Deliverables

- [Required] Typed layout engine (order/pinning/overflow) with pure functions and versioned config validation
- [Required] Workspace-wide setting to persist Room Toolbox configuration
- [Required] Administration UI under Administration > Settings > Layout > Room Header for reordering and pinned/featured actions
- [Required] Config consumption with useSetting and dynamic structure updates with useSettingStructure

- [Required] Per-user Feature Preview integration (toggle to enable new engine)
- [Required] Unit tests (Jest/RTL) for layout engine and admin UI
- [Required] E2E tests (Playwright) + accessibility checks (axe) for preview flow and header rendering
- [Required] Complete documentation (config behavior, how to add new actions, compatibility rules)
- [Optional] Expanded support for app-action pinning/ordering (if required and within schedule)

Timeline

Week	Period	Activity	Deliverable
—	Before Program …	Architecture study (RoomToolbox, roomActionHooks, Feature Preview, tests, Storybook)	Environment setup + architecture notes
1-2	Community Bond…	Design discussions with mentors; close v1 scope (core vs app actions), setting format, fallback rules	Design document
3-4	Coding	Implement types + layout engine (pure functions) and engine unit tests	Tested engine v1
5-6	Coding	Admin UI with Fuselage + setting persistence + UI tests	Functional admin config
7	Coding	Per-user Feature Preview integration + connect engine to RoomToolbox rendering	*Preview* ON/OFF working
8	Midterm	Mentor review; adjust UX/compatibility decisions; assess testing	Midterm evaluation

		gaps	
9-10	Coding	Coding | Handle edge cases (unknown actions, permissions, overflow), refine app-action compatibility	Robust behavior
11	Coding	Strong test cycle (E2E + axe) + bug and flakiness fixes	Complete test suite
12	Coding	Storybook stories + technical and admin documentation	Final docs + stories
13	Final	Buffer + stabilization + final report	Completed project

Milestone Acceptance Criteria

- Engine v1 Milestone: documented rules and unit tests covering main scenarios and edge cases
- Admin Settings Milestone: configuration saved and correctly reflected in UI under Administration > Settings > Layout > Room Header
- Preview Integration Milestone: ON/OFF switching via /account/feature-preview changes behavior as expected without side effects
- Final Quality Milestone: stable E2E + axe suite and sufficient long-term maintenance documentation

Related Work

- Feature Preview in Rocket.Chat: /account/feature-preview already exists and stores per-user preferences; E2E + axe accessibility tests already exist
- Room Toolbox and header actions: actions come from roomActionHooks and apps/plugins; current layout uses featured + visible-item limit + overflow
- Progressive delivery / feature flags: this proposal follows gradual rollout using the existing Feature Preview mechanism, avoiding immediate global activation.

Final Notes

This proposal focuses on a robust and incremental MVP to improve Room Header efficiency without breaking the current user experience. The combination of versioned settings, per-user rollout, and quality coverage (unit, E2E, a11y, and

Storybook) reduces technical risk and accelerates mentor review.

About Me

I am a Computer Science student and developer focused on accessibility, UX, and code quality. I have experience with TypeScript/React and with unit testing (RTL/Jest) and E2E testing (Playwright), and I actively contribute improvements in i18n and accessibility.

Contributions to the Organization

Merged:

- PR #38662 - fix(i18n): use correct i18n key for workspace confirmation (https://github.com/RocketChat/Rocket.Chat/pull/38662)

Open:

- PR #38738 - fix: improve label association in omnichannel unit fields (https://github.com/RocketChat/Rocket.Chat/pull/38738)
- PR #38666 - fix: improve MultiSelectSettingInput accessibility with aria-labelledby (https://github.com/RocketChat/Rocket.Chat/pull/38666)
- PR #38627 - refactor(ui-client): improve Callbacks class typing v2 (https://github.com/RocketChat/Rocket.Chat/pull/38627)
- PR #38626 - refactor(ui-contexts): improve ActionManager and AvatarUrl typing (https://github.com/RocketChat/Rocket.Chat/pull/38626)
- PR #38595 - chore: improve typing for ContextualbarResizable resize handler (https://github.com/RocketChat/Rocket.Chat/pull/38595)
- PR #38594 - chore: improve GenericMenu TypeScript typing (https://github.com/RocketChat/Rocket.Chat/pull/38594)
- PR #38569 - chore: improve typing for addCustomField using JsonValue (https://github.com/RocketChat/Rocket.Chat/pull/38569)

Technical Skills

- TypeScript / JavaScript: refactoring in large codebases; strong typing; context integration
- React: hooks, context, functional components; composition patterns
- Testing: Jest/RTL (unit), Playwright (E2E), axe-core (A11y)
- Accessibility: hands-on WAI-ARIA practice and automated validation

Availability

1. **Hours per week:** 30-35.
2. **Other commitments:** Computer Science degree (flexible schedule).
3. **Unavailable periods:** occasional exam windows (1-3 days, communicated in advance).
4. **Compensation plan:** recover unavailable hours in adjacent weeks to keep milestones on track.

Communication Plan

1. **Status reports:** weekly updates via Rocket.Chat and GitHub.
2. **Preferred channel:** Rocket.Chat Contributors Workspace and GitHub discussions.
3. **Call availability:** 1 to 3 sync calls per week (UTC-3).
4. **Commitment:** proactive communication on blockers, risks, and rollout progress.